Quiz: Frontend Engineering Onboarding: Advanced React & Modern Standards

Module 1 Quiz

1. Where are reusable packages and shared configurations located in our monorepo?

A. In the /apps directory
B. In the /packages directory
C. In the root node_modules folder
D. In the /config directory

2. Which tool is used to enforce code formatting and linting during the pre-commit stage?

A. Turborepo
B. Husky and lint-staged
C. Volta
D. Tailwind CSS

Module 2 Quiz

3. What pattern is recommended for managing implicit state among a group of related UI components?

A. Prop Drilling Pattern
B. Higher-Order Components
C. Compound Components Pattern
D. Redux State Pattern

4. Why is Zustand preferred over React Context for high-frequency global state updates?

A. Zustand does not support TypeScript.
B. React Context can trigger unnecessary re-renders across the entire component tree.
C. Zustand is built directly into the React core library.
D. React Context cannot handle asynchronous actions.

Module 3 Quiz

5. Which library is our standard for managing and caching server state?

A. Zustand
B. Axios

C. TanStack Query
D. Zod

6. How do we validate external API payloads at the application boundary?

A. Using TypeScript 'any' types
B. Using Zod schemas for runtime validation
C. Using Axios interceptors only
D. Using Jest assertions

Module 4 Quiz

7. What is the preferred way to query elements when writing integration tests with React Testing Library?

A. By querying CSS class names directly
B. By using internal component state values
C. By accessible roles (e.g., getByRole)
D. By database IDs

8. Which tool do we use to automate accessibility audits during integration testing?

A. Jest-axe
B. ESLint
C. Prettier
D. React DevTools

Module 5 Quiz

9. What should you do before applying optimization techniques like useMemo or useCallback?

A. Apply them to every component by default
B. Profile the application using React DevTools to identify actual bottlenecks
C. Disable CI/CD pipelines to speed up local builds
D. Convert all components into class components

10. What commit standard do we follow to maintain structured and automated changelogs?

A. Ad-hoc Commit Messages
B. Conventional Commits
C. GitHub Issue Linking only
D. Git Flow Commits

Answer Key

Q1: B - Our monorepo structure separates deployable applications in /apps and reusable libraries, design tokens, and shared configurations in /packages.

Q2: B - Husky and lint-staged work together to run ESLint and Prettier on staged files before a commit is finalized.

Q3: C - The Compound Components pattern allows parent and child components to share state implicitly using React Context, making the JSX clean and flexible.

Q4: B - React Context can cause performance bottlenecks because any update to the context value forces all consuming components to re-render. Zustand avoids this with selector-based subscriptions.

Q5: C - TanStack Query (formerly React Query) is our standard library for fetching, caching, and synchronizing server state.

Q6: B - We use Zod schemas to perform runtime validation on API payloads, ensuring that unexpected backend changes do not cause silent frontend failures.

Q7: C - React Testing Library recommends querying elements by their accessible roles to mimic how assistive technologies and real users interact with the UI.

Q8: A - We integrate jest-axe into our testing suite to run automated accessibility checks and catch common WCAG violations.

Q9: B - Premature optimization can complicate code. You should always profile the application first to ensure memoization is targeting an actual performance bottleneck.

Q10: B - We follow the Conventional Commits standard to ensure our commit history is structured, readable, and compatible with automated release tooling.